

The Claude Code Starter Pack

Everything I wish someone had handed me on day one. The setup, the config file that does most of the work, and the eleven commands worth learning.

Bryan Collins · bryancollins.com

Who this is for

You are not a professional developer. You can read code but you would not claim to write it. You want to build and ship real things anyway.

That is exactly the position I was in. I now run seven live sites from the terminal.

1. Setup, start to finish

```
npm install -g @anthropic-ai/claude-code
cd your-project
claude
```

Three commands. The first run asks you to sign in.

Two things to do immediately:

Pick your permission mode. Claude Code asks before it edits files or runs commands. Leave that on while you are learning. You want to see what it is about to do.

Run it inside a git repo. If you are not in one, `git init` first. Git is your undo button. Without it, a bad edit is permanent; with it, `git checkout .` undoes everything since your last commit. This matters more than any other tip in this document.

2. The CLAUDE.md file

This is the highest-leverage thing you will do. `CLAUDE.md` sits in your project root and loads into every session automatically. It is how you stop re-explaining your project every morning.

Copy this, save it as `CLAUDE.md`, fill in the blanks:

```
# PROJECT: <name>

**Stack:** <e.g. Astro 4, Tailwind, deployed on Netlify>
**Live at:** <url>

## What this project is
<Two sentences. What it does and who it is for.>

## How to run it
- Dev server: `npm run dev`
- Build: `npm run build`
- Deploy: push to `main`, <host> builds automatically

## Where things live
- Pages: `src/pages/`
- Components: `src/components/`
- Data: `src/data/`
- Styles: <one line>

## Rules
- Never upgrade a framework or dependency unless I ask.
- Run the build locally before pushing. A failing build in CI is not acceptable.
- Match the style of surrounding code. Do not introduce a new pattern.
- Do not add features I did not ask for.
- Ask before deleting anything.

## Things that have bitten me
<Add a line here every time something breaks. This section becomes
the most valuable part of the file within a month.>
```

That last section is the one that compounds. Every time you hit a weird failure, write one line. Six weeks later the file is worth more than the code.

3. The eleven commands worth learning

Command	What it does
<code>/init</code>	Reads your codebase and writes a first-draft <code>CLAUDE.md</code>
<code>/clear</code>	Wipes the conversation. Use it between unrelated tasks
<code>/compact</code>	Summarises a long session so you can keep going
<code>/model</code>	Switch model mid-session
<code>/config</code>	Settings, including permission mode
<code>/help</code>	Everything available right now
<code>/review</code>	Reviews your current changes before you commit
<code>/cost</code>	What this session has used
<code>#</code>	Type <code>#</code> then a fact to save it to <code>CLAUDE.md</code> permanently
<code>!</code>	Prefix a shell command to run it and drop the output into context
<code>Esc</code>	Interrupt. Use it the moment it goes the wrong way

`Esc` is the one people underuse. Do not sit and watch it build the wrong thing out of politeness.

4. Five habits that separate good sessions from bad ones

Say what "done" looks like. Not "fix the header". Instead: "the header should stay fixed on scroll and collapse to a hamburger under 768px. Tell me when the build passes."

One task per session. `/clear` between unrelated jobs. A long session carrying three abandoned tasks makes worse decisions than a fresh one.

Make it show you. "Run the build and show me the output" beats "does it work?" every time. A claim is not evidence.

Commit early, commit often. Before any change you are unsure about: `git add -A && git commit -m "before X"`. Cheap insurance.

Give it the error, not your summary of the error. Paste the whole stack trace. Your paraphrase drops the line that mattered.

5. The three mistakes I made

I let it upgrade things. An unprompted Tailwind major-version bump cost me a day. Hence the rule in the template above.

I trusted "fixed" without a check. A change is not fixed until something that failed before passes now. Ask for the before and after.

I skipped `CLAUDE.md` for a month. I re-explained the same project every single morning. Twenty minutes to write the file would have saved me hours.

6. A first project that actually teaches you something

Do not start with your real site. Do this instead, it takes an evening:

1. `mkdir portfolio && cd portfolio && git init`
2. `claude`
3. "Build me a one-page personal site. Plain HTML and CSS, no framework, no build step. My name is X, I do Y. Include a short bio and three links."
4. "Now deploy it to Netlify and give me the URL."

You will hit a real problem somewhere in step 4. Solving it is the point.

7. Skills: the part most people never find

A skill is a reusable instruction set you invoke with a slash command. Write

`.claude/commands/deploy.md` describing your deploy steps, and `/deploy` runs them the same way every time.

I keep mine public, free, MIT licensed:

github.com/bryancollins99/agent-skills

Clone it, copy what is useful into your own `.claude/commands/`.

What next

I publish what I build, including what breaks, at bryancollins.com and on YouTube three times a week.

© Bryan Collins · bryancollins.com